Multimedia Signal Processing - 2nd module

Prof. Marco Tagliasacchi

February, 22 2011

1 Question - 10 pts

Consider a coding scheme that needs to transmit the position of a mouse pointer and mouse click events. You are given the following specifications:

- The screen resolution is 640×480 pixels.
- The coordinates (x, y) are expressed as integer values.
- There are three types of events e: left click (L), right click (R), idle (I).
- The mouse data $(x, y, e)_t$, where t denotes a time index, is updated at a frequency of 60Hz. An algorithmic delay of 1/30 seconds is tolerable.
- The following (rather unrealistic, though) statistics are given

$$\begin{aligned} p(L) &= 0.10 \\ p(R) &= 0.05 \end{aligned} \tag{1}$$

- Each triple $(x, y, e)_{t_i}$ is assumed (for simplicity) statistically independent from $(x, y, e)_{t_j}$, $t_i \neq t_j$.

Add additional assumptions, whenever needed, and do the following.

1. Design a lossless coding scheme (e.g. based on Huffman coding), and compute the overall average bitrate.
2. Discuss how to modify your design in order to take into account statistical dependencies between $(x, y, e)_{t_i}$ and $(x, y, e)_{t_j}$, $t_i \neq t_j$.

Solution

Let's consider how to encode the information about mouse location (x, y). Each coordinate of the mouse location can be modeled as a discrete source with alphabets, respectively, of size $M_x = 640$, $M_y = 480$. It is reasonable to assume that

- x and y are statistically independent;
- The symbols of each source are equally likely. That is $p(x) = 1/640$, $x \in 1, \ldots, 640$ and $p(y) = 1/480$, $y \in 1, \ldots, 480$ and

Based on that, and on the hypothesis that $(x, y, e)_{t_i}$ is statistically independent from $(x, y, e)_{t_j}$, $t_i \neq t_j$, a lower bound on the average bitrate is

$$R_x \geq H(x) = \log_2 640 = 9.32 \text{ bps} \tag{2}$$

$$R_y \geq H(y) = \log_2 640 = 8.90 \text{ bps} \tag{3}$$

The simplest strategy is to lossless code each symbol independently using, respectively

$$R_x = \lceil \log_2(640) \rceil = 10 \text{ bps} \tag{4}$$

$$R_y = \lceil \log_2(480) \rceil = 9 \text{ bps} \tag{5}$$

Grouping symbols is not expected to help much, as the number of symbols in the alphabet is large.

Let's consider how to encode the information about mouse events. Given the source statistics, and the hypothesis of statistical independence, a lower bound is

$$R_e = -p(L)\log 2p(L) - p(R)\log 2p(R) - p(I)\log 2p(I) = 0.7476 \text{ bps} \tag{6}$$

where $p(I) = 1 - p(L) - p(R) = 0.85$.
The simplest solution is to encode each symbol independently, e.g. based on Huffman coding. A possible Huffman code is the following (generated by assigning 0 to the left branch, 1 to the right branch, and putting the symbol with the least probability to the left).

$$\begin{aligned}(L) &\to 00\\ (R) &\to 01\\ (I) &\to 1\end{aligned} \tag{7}$$

The average code length is

$$L = 2 \cdot p(L) + 2 \cdot p(R) + 1 \cdot p(I) = 1.15 \text{ bps} \tag{8}$$

We can design a more efficient coding scheme based on Huffman coding. Indeed, since the maximum tolerable delay is 1/30 seconds, and the source emits symbols every 1/60 seconds, we could group $N = 2$ symbols together. We can form $3^2 = 9$ supersymbols, where joint probabilities are given by the product of the marginals (due to statistical independence)

$$\begin{aligned}p(LL) &= 0.010\\ p(LR) &= 0.005\\ p(LI) &= 0.085\\ p(RL) &= 0.005\\ p(RR) &= 0.0025\\ p(RI) &= 0.0425\\ p(IL) &= 0.085\\ p(IR) &= 0.0425\\ p(II) &= 0.7225\end{aligned} \tag{9}$$

A possible Huffman code is the following (generated by assigning 0 to the left branch, 1 to the right branch, and putting the symbol with the least probability to the left).

$$\begin{aligned}(LL) &\rightarrow 011100\\ (LR) &\rightarrow 01110111\\ (LI) &\rightarrow 00\\ (RL) &\rightarrow 0111010\\ (RR) &\rightarrow 01110110\\ (RI) &\rightarrow 01111\\ (IL) &\rightarrow 010\\ (IR) &\rightarrow 0110\\ (II) &\rightarrow 1\end{aligned} \tag{10}$$

The average code length is

$$L = (6{\cdot}p(LL)+8{\cdot}p(LR)+2{\cdot}p(LI)+7{\cdot}p(RL)+8{\cdot}p(RR)+5{\cdot}p(RI)+3{\cdot}p(IL)+4{\cdot}p(IR)+1{\cdot}p(II))/2 = 1.6850/2 = 0.8425 \text{ bps} \tag{11}$$

The overall bitrate is

$$(R_x + R_y + R_e) \cdot 60 = (10 + 9 + 0.8425) \cdot 60 = 1190 \text{ bps} \tag{12}$$

Statistical dependencies between $(x, y, e)_{t_i}$ and $(x, y, e)_{t_j}$, $t_i \neq t_j$, might involve both mouse location and mouse events

- Mouse location: Consider the source x (an equivalent discussion holds for y). Source memory can be tacked by
  - Entropy coding groups of symbols. However, the delay constraint enforces that $N \geq 2$.
  - Create a new source $d = x_t - x_{t-1}$. The alphabet of d is larger (i.e. $-640, 640$). However, due to temporal dependencies, we can expect that $H(d) \ll H(x)$. From an engineering point of view, this solution is probably preferred.
- Mouse events: We could adopt the same design as above, where temporal dependencies induce values of the joint probabilities that cannot simply derived from the marginals, but they have to be estimated from data. More sophisticated techniques (e.g. run-level coding) cannot be applied due to the delay constraint.

2 Question - 5 pts

Briefly illustrate transform coding. In particular, you need to discuss:

1. The application scenarios where it might be adopted;
2. How optimal bit allocation is performed;
3. The relationship with vector quantization.

Solution

See slide set `MMSP_1.4_Transform coding`.

3 Questions - 5 pts (each answer can be either TRUE or FALSE)

1. Consider an arithmetic coder and a discrete source X
   - [T] [F] The average codelength L is such that $L \leq H(X) - 1$
     FALSE. In general $L \leq H(X) + 2/N$, where N is the number of encoded symbols.
   - [T] [F] Source memory can be exploited by adaptively tracking the source statistics.
     TRUE. Arithmetic coders allow to adapt both the encoder and the decoder to the source statistics at a small computational cost.
   - [T] [F] The arithmetic coder can be used only if X is a binary source
     FALSE. Arithmetic coding can be applied to any discrete source, regardless of the alphabet size. In some applications, though, source symbols are binarized before arithmetic coding. This is done primarily for ease of implementation and computational efficiency.
2. Consider a speech coding system
   - [T] [F] The short term predictor is useful when coding unvoiced speech
     TRUE. The short term predictor captures the spectral envelope of both voiced and unvoiced speech.
   - [T] [F] Quantization of reflection coefficients leads to instability at the decoder
     FALSE. Reflection coefficients k_i, $i = 1, \ldots, p$ are constrained in the $(-1, 1)$ range. If the quantizer is designed in such a way that quantization symbols fall in the same range, stability is guaranteed
   - [T] [F] The number of zeros of the whitening filter $A(z)$ depends on the number of formants below 4 kHz
     TRUE. The whitening filter $A(z)$ (corresponding to either the long or short-term predictor, or both) is an all-zero filter. The number of zeros depends on the number of resonant peaks (formants) below 4kHz
3. Consider a block-based image coding system
   - [T] [F] Zonal quantization adaptively allocates the bitrate across the image
     FALSE. With zonal quantization, DCT coefficient statistics are collected beforehand at training time and bit allocation is performed accordingly. Thus, the number of bits per block is fixed across the image.
   - [T] [F] Decreasing the block size, the coding efficiency decreases
     TRUE. Larger block sizes allow to exploit inter-pixel correlations at longer distances. The coding gain is expected to decrease when the block size decreases.
   - [T] [F] DC coefficients in different blocks are statistically independent
     FALSE. DC coefficients capture the average values of the blocks. Hence, in smooth areas, DC coefficients of neighboring blocks are statistically dependent.
   - [T] [F] The PSNR between the original and the compressed image increases, as the bitrate increases
     TRUE. Allocating more bits allows finer quantization step sizes. Therefore MSE decreases and PSNR increases

4 Question (Matlab) - 12 pts

A wireless multimedia sensor needs to send the greyscale image `marshall.pgm` to the base station. A radio channel is used at a data rate of 256 kbps.

- Find the transmission time of the uncompressed image.
- Find the minimum transmission time of the entropy-coded image (you can model pixel values as symbols of a discrete memoryless source)
- Given that the image needs to be delivered in less than 1 second, implement a PCM coding scheme that satisfy this constraint. Compute the PSNR of the compressed image with respect to the original.
- Implement a simple JPEG baseline (8×8 blocks, quality factor $q = 1$) using the DCT quantization matrix `Qjpeg.mat`. Compute the transmission time and the PSNR in this case. (Hint: you can estimate the rate using the ρ-domain model: $R = \theta(1 - \rho)$ where ρ is the fraction of quantized DCT coefficients equal to zero and $\theta = 5$).

Solution

```
clear all
close all
clc

I = double(imread('marshall.pgm'));
[M N] = size(I);
c = 256*1024; %[bps]

%raw image transmission
t = (M*N*8)/c;
disp(['Raw image transmission time: ' num2str(t,2) ' seconds']);

%lossless coding
h = hist(I(:),[0:255]);
p = h./sum(h);
p = p(p~=0);
h = -sum(p.*log2(p));
disp(['Entropy: ' num2str(h,2) ' bpp']);
t_h = (M*N*h)/c;
disp(['Lossless coded image transmission time: ' num2str(t_h,2) ' seconds']);


%PCM coding

%R <= T [sec] * c [bps] / (M*N)
R_pcm = floor(1*c/(M*N));
Q = 255/2^R_pcm;
I_pcm = Q*round(I/Q);
MSE = sum(sum((I-I_pcm).^2))/(M*N);
PSNR_pcm = 10*log10(255^2/MSE);

t_pcm = (M*N*R_pcm)/c;
disp(['PCM coded image transmission time: ' num2str(t_pcm,2) ' seconds. PSNR: ' num2str(PSNR_pcm,3) ' dB']);

%Transform coding
load Qjpeg.mat
iq = 1;
k=8;


%jpeg baseline
temp_rho = 0;
for ir=1:M/k
    for ic=1:N/k
        %select 8x8 block
        yb = I((ir-1)*k + 1:ir*k, (ic-1)*k + 1:ic*k);
        %2D dct
        ybdct = dct2(yb);
        %threshold quantization
        ybdct_q = round(ybdct./(iq*QJPEG));
        temp_rho = temp_rho + length(find(ybdct_q==0));
        %inverse dct and dequantization
        I_dct((ir-1)*k + 1:ir*k, (ic-1)*k + 1:ic*k) = idct2(ybdct_q.*QJPEG);
    end
end
MSE = sum(sum((I-I_dct).^2))/(M*N);
PSNR_dct = 10*log10(255.^2/MSE);
rho = temp_rho/(M*N);


%estimate rate
theta = 5;
```

```
R_dct = theta*(1-rho);
t_dct = (M*N*R_dct)/c;

disp(['DCT coded image transmission time: ' num2str(t_dct,2) ' seconds. PSNR: ' num2str(PSNR_dct,3) ' dB']);

subplot(131);imshow(I,[]);
subplot(132);imshow(I_pcm,[]);
subplot(133);imshow(I_dct,[]);
```